

GV Nguyễn Đức Kiên

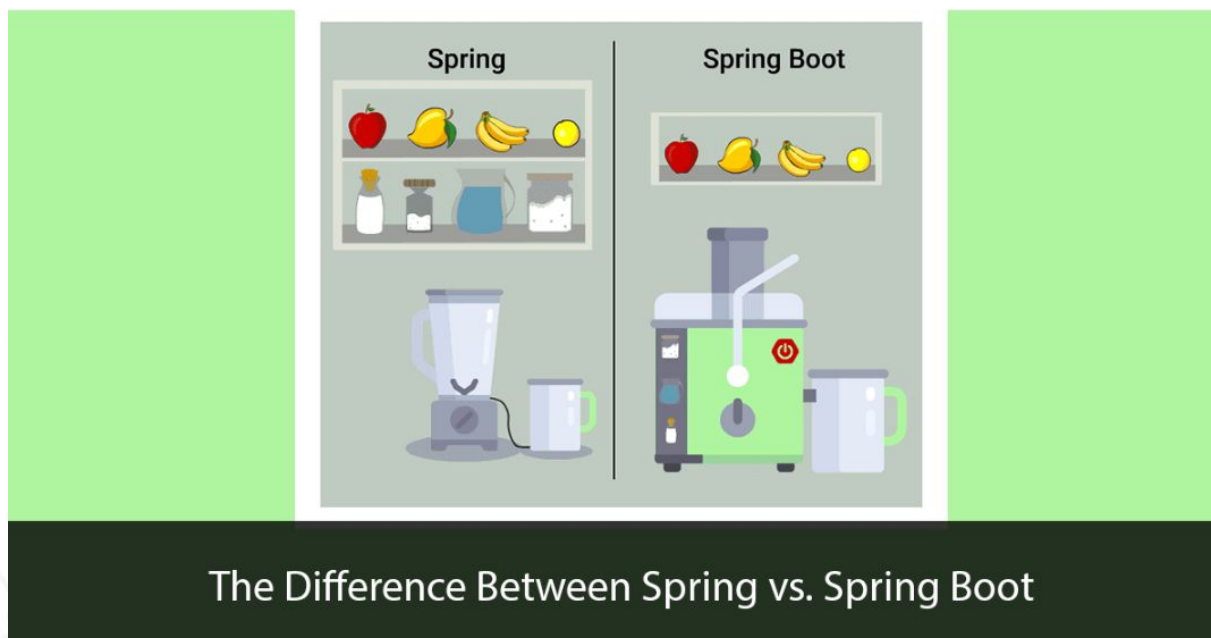


Nội dung

1. Giới Thiệu về Spring Boot
2. Các Cải Tiến của Spring Boot so với Spring MVC
3. Cơ Chế Hoạt Động của Spring Boot
4. Kiến Trúc của Spring Boot, Mô Hình 3 Layer
5. Các Bước Tạo Một Project Spring Boot

1. Giới Thiệu về Spring Boot

- Spring Boot là một framework giúp phát triển ứng dụng nhanh chóng, giảm bớt sự phức tạp của cấu hình ứng dụng Spring truyền thống. Spring Boot cung cấp các công cụ để tự động hóa cấu hình và giúp các nhà phát triển dễ dàng triển khai ứng dụng vào môi trường sản xuất.



1. Giới Thiệu về Spring Boot

Lợi ích của Spring Boot

- ❑ **Giảm thiểu cấu hình:** Spring Boot tự động cấu hình các thành phần trong ứng dụng dựa trên các thư viện mà bạn đã thêm vào project.
- ❑ **Không cần file web.xml:** Spring Boot không yêu cầu cấu hình tệp web.xml, giúp đơn giản hóa việc triển khai ứng dụng web.
- ❑ **Hỗ trợ tích hợp sẵn:** Spring Boot tích hợp sẵn với các công cụ như Maven hoặc Gradle để dễ dàng quản lý phụ thuộc (dependencies).
- ❑ **Ứng dụng có thể chạy độc lập:** Spring Boot cho phép ứng dụng chạy độc lập mà không cần một máy chủ ứng dụng bên ngoài (ví dụ như Tomcat hay Jetty).

2) Các Cải Tiến của Spring Boot so với Spring MVC

- **Cấu hình tự động:** Một trong những điểm mạnh của Spring Boot là khả năng tự động cấu hình ứng dụng. Khi bạn thêm một thư viện, Spring Boot sẽ tự động cấu hình các bean cần thiết mà không cần bạn phải viết cấu hình thủ công trong các file như web.xml hay applicationContext.xml.
- **Khởi động nhanh hơn:** Ứng dụng Spring Boot có thể khởi động nhanh chóng nhờ vào cấu hình tự động và việc sử dụng các máy chủ nhúng. Điều này khác với Spring MVC truyền thống, nơi bạn cần cấu hình và triển khai ứng dụng trên một máy chủ ngoài (ví dụ: Tomcat).
- **Ứng dụng độc lập:** Spring Boot cho phép bạn phát triển các ứng dụng web hoặc microservices có thể chạy độc lập mà không cần một máy chủ web ngoài.

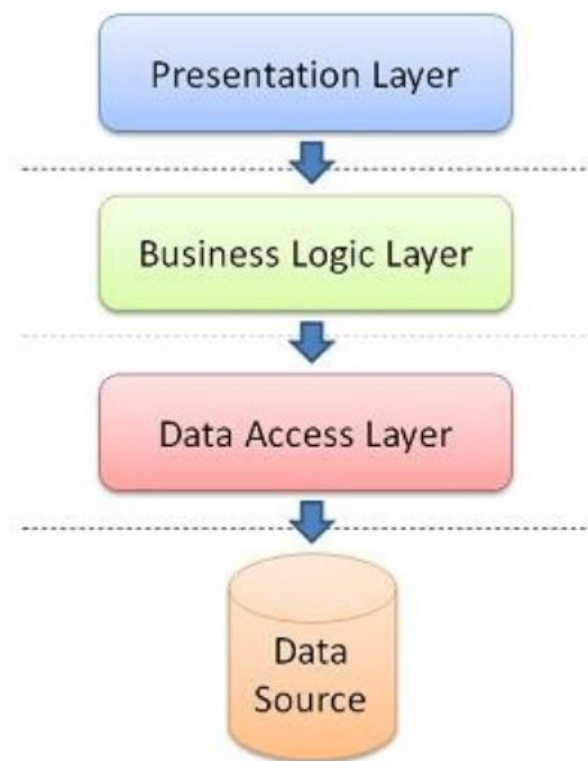
3) Cơ Chế Hoạt Động của Spring Boot

Spring Boot hoạt động theo một cơ chế tự động cấu hình mạnh mẽ, giúp đơn giản hóa quy trình phát triển ứng dụng.

- ❑ **Auto Configuration (Cấu hình tự động):** Spring Boot tự động cấu hình các thành phần của ứng dụng (ví dụ: datasource, session, JPA, security, etc.) dựa trên các thư viện mà bạn đã thêm vào project.
- ❑ **Spring Boot Starters:** Các starter là các gói thư viện được cấu hình sẵn, giúp dễ dàng thêm các tính năng vào ứng dụng.
Ví dụ:
 - spring-boot-starter-web: Cho ứng dụng web sử dụng Spring MVC.
 - spring-boot-starter-data-jpa: Dành cho ứng dụng sử dụng JPA với cơ sở dữ liệu.
 - spring-boot-starter-security: Cho các ứng dụng yêu cầu bảo mật.Spring Boot Auto
- ❑ **Configuration:** Khi bạn thêm một dependency vào ứng dụng (ví dụ: spring-boot-starter-data-jpa), Spring Boot sẽ tự động cấu hình các bean cần thiết mà không yêu cầu bạn phải viết mã cấu hình.
- ❑ **Embedded Servers:** Spring Boot hỗ trợ các máy chủ nhúng như Tomcat, Jetty, hoặc Undertow, giúp bạn không phải cấu hình máy chủ bên ngoài.

4) Kiến Trúc của Spring Boot, Mô Hình 3 Layer

- Spring Boot thường được xây dựng theo mô hình kiến trúc ba lớp (3-tier architecture)



4) Kiến Trúc của Spring Boot, Mô Hình 3 Layer

1. Presentation Layer (Tầng Trình Bày):

- ☐ Chịu trách nhiệm xử lý các yêu cầu từ người dùng và trả lại kết quả.
- ☐ Trong Spring Boot, tầng này thường được xử lý bằng các controller (@RestController hoặc @Controller).
- ☐ Code ví dụ:

```
14
15  @RestController  @kiennd25
16  @RequestMapping("/api/claims")
17  public class ClaimResourceController {
18
19      @Autowired
20      private ClaimService claimService;
21
22      @GetMapping("/id")  @kiennd25
23      public ClaimDTO getById(@PathVariable Long id) { return claimService.getById(id); }
26
```

4) Kiến Trúc của Spring Boot, Mô Hình 3 Layer

2. Business Layer (Tầng Xử Lý Nghiệp Vụ):

- ☐ Tầng này chứa logic nghiệp vụ của ứng dụng.
- ☐ Các lớp @Service thường thực hiện các thao tác nghiệp vụ trong tầng này.
- ☐ Code ví dụ:

```
@Service  @kiennd25
public class ClaimServiceImpl implements ClaimService {

    @Autowired
    private ClaimRepository claimRepository;

    @Autowired
    private ClaimMapper claimMapper;

    @Override  @kiennd25
    public ClaimDTO getById(Long id) {
        ClaimEntity claimEntity = claimRepository.findById(id)
            .orElseThrow(() -> new RuntimeException("Claim not found"));
        return claimMapper.toDTO(claimEntity);
    }
}
```

4) Kiến Trúc của Spring Boot, Mô Hình 3 Layer

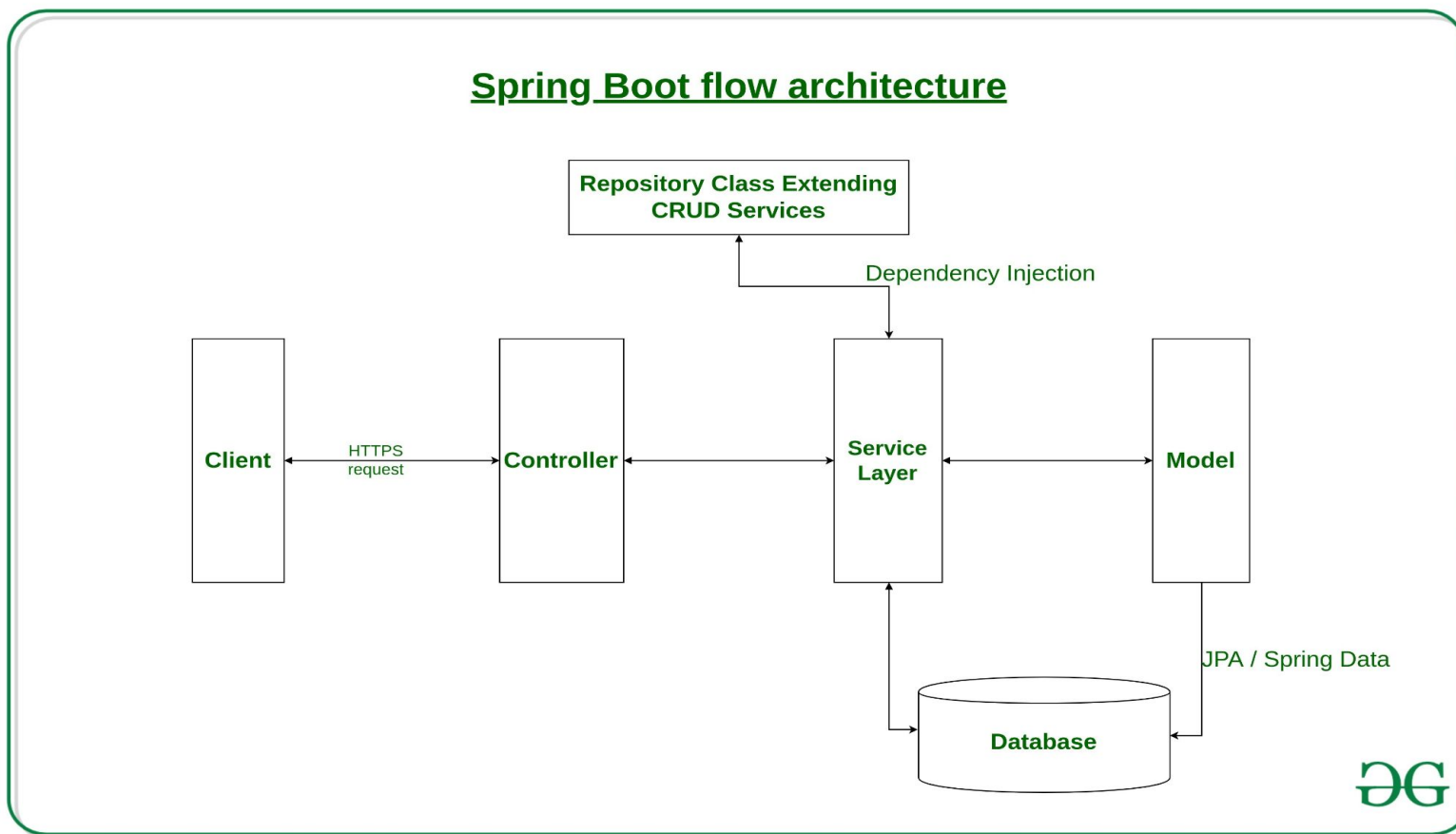
3. Persistence Layer (Tầng Lưu Trữ):

- ☐ Tầng này tương tác với cơ sở dữ liệu, thực hiện các thao tác CRUD.
- ☐ Các lớp repository trong Spring Boot thường được đánh dấu bằng `@Repository` và sử dụng Spring Data JPA hoặc các phương thức SQL tùy chỉnh.
- ☐ Code ví dụ

```
14  @Repository 2 usages 1 kiennnd25
15
16  public interface ClaimRepository extends JpaRepository<ClaimEntity, Long> {
17
18      @Query("SELECT c FROM ClaimEntity c " + 1 usage 1 kiennnd25
19              "WHERE (:#{#request.claimCode} = ' ' OR LOWER(c.code) LIKE LOWER(CONCAT('%', :#{#request.claimCode}, '%')) " +
20              "AND (:#{#request.statusCode} = ' ' OR c.claimStatusEntity.code = :#{#request.statusCode}) " +
21              "AND (:#{#request.fromDateSearch} IS NULL OR c.claimDate >= :#{#request.fromDateSearch}) " +
22              "AND (:#{#request.toDateSearch} IS NULL OR c.claimDate <= :#{#request.toDateSearch}) " +
23              "AND (c.deleted=false OR c.deleted is NULL) ")
24      Page<ClaimEntity> findByCondition(@Param("request") ClaimRequest request, Pageable pageable);
```

5) Giải Thích Luồng Đi của Mô Hình Spring Boot

- Từ mô hình 3 layer phía trên sẽ triển khai thực tế theo luồng như sau



5) Giải Thích Luồng Đi của Mô Hình Spring Boot

1. Client

- **Vai trò:** Đây là người dùng hoặc hệ thống gửi yêu cầu HTTP đến ứng dụng Spring Boot. Yêu cầu có thể đến từ trình duyệt, ứng dụng di động, hay hệ thống khác.
- **Các phương thức HTTP:** Yêu cầu có thể là các phương thức HTTP như GET, POST, PUT, DELETE, v.v.



5) Giải Thích Luồng Đi của Mô Hình Spring Boot

- **2. Controller**

- **Vai trò:** Đây là lớp trung gian nhận và xử lý các yêu cầu HTTP từ Client.

- **Cách hoạt động:**

- ☐ Controller nhận yêu cầu từ client (ví dụ: thông qua một URL cụ thể).
- ☐ Nó sử dụng các phương thức từ các lớp service để xử lý nghiệp vụ và trả lại phản hồi cho client.
- ☐ Controller được đánh dấu với annotation `@RestController` hoặc `@Controller`.

```
14
15  ✓ @RestController  kiennnd25
16  @RequestMapping("/api/claims")
17  public class ClaimResourceController {
18
19      @Autowired
20      private ClaimService claimService;
21
22      @GetMapping("/id")  kiennnd25
23      public ClaimDTO getById(@PathVariable Long id) { return claimService.getById(id); }
24
25
26
```

5) Giải Thích Luồng Đi của Mô Hình Spring Boot

3. Service Layer

- **Vai trò:** Tầng dịch vụ xử lý các logic nghiệp vụ của ứng dụng.
- **Cách hoạt động:**
 - ❑ Sau khi Controller nhận yêu cầu, nó sẽ gọi các phương thức từ Service Layer để thực hiện các thao tác nghiệp vụ.
 - ❑ Service Layer nhận dữ liệu từ Repository hoặc thực hiện tính toán/phân tích dữ liệu.
 - ❑ Đây là nơi chứa các logic phức tạp, tính toán hoặc xử lý các yêu cầu từ Controller.
 - ❑ Service Layer giúp tách biệt các logic nghiệp vụ khỏi Controller, giữ cho mã nguồn dễ bảo trì.

```
@Service  kiennnd25
public class ClaimServiceImpl implements ClaimService {

    @Autowired
    private ClaimRepository claimRepository;
```

5) Giải Thích Luồng Đi của Mô Hình Spring Boot

4. Repository (Repository Class Extending CRUD Services)

- **Vai trò:** Đây là tầng tương tác với cơ sở dữ liệu.
- **Cách hoạt động:**
 - ❑ Repository sử dụng JPA (Java Persistence API) hoặc Spring Data để thực hiện các thao tác CRUD (Create, Read, Update, Delete) trên cơ sở dữ liệu.
 - ❑ Repository tương tác trực tiếp với bảng dữ liệu trong cơ sở dữ liệu.
 - ❑ Spring Data cung cấp các interface như JpaRepository hoặc CrudRepository để thực hiện các thao tác này mà không cần viết nhiều mã.

```
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import org.springframework.data.domain.Pageable;

public interface ClaimRepository extends JpaRepository<ClaimEntity, Long> {

    @Query("SELECT c FROM ClaimEntity c " +
        "WHERE (:#{request.claimCode} = '' OR LOWER(c.code) LIKE LOWER(CONCAT('%', :#{request.claimCode}, '%')) " +
        "AND (:#{request.statusCode} = '' OR c.claimStatusEntity.code = :#{request.statusCode}) " +
        "AND (:#{request.fromDateSearch} IS NULL OR c.claimDate >= :#{request.fromDateSearch}) " +
        "AND (:#{request.toDateSearch} IS NULL OR c.claimDate <= :#{request.toDateSearch}) " +
        "AND (c.deleted=false OR c.deleted is NULL) ")
    Page<ClaimEntity> findByCondition(@Param("request") ClaimRequest request, Pageable pageable);
}
```

5) Giải Thích Luồng Đi của Mô Hình Spring Boot

5. Model (Entity)

- **Vai trò:** Lớp mô hình (Model) là các đối tượng dữ liệu, tương ứng với bảng trong cơ sở dữ liệu.
- **Cách hoạt động:**
 - ❑ Mô hình đại diện cho các đối tượng mà ứng dụng thao tác, ví dụ như Claim, User, Order, v.v.
 - ❑ Các lớp mô hình này thường được đánh dấu với annotation `@Entity`, xác định rằng lớp này là một đối tượng JPA sẽ được ánh xạ vào một bảng trong cơ sở dữ liệu.

```
@Entity 12 usages  🧑 kienn25
@Table(name = "claim")
@Getter
@Setter
public class ClaimEntity extends BaseEntity {

    private String code;

    @ManyToOne
    @JoinColumn(name = "customer_id")
    private CustomerEntity customerEntity;
```

5) Giải Thích Luồng Đi của Mô Hình Spring Boot

6. Database

- **Vai trò:** Là nơi lưu trữ dữ liệu của ứng dụng. Cơ sở dữ liệu có thể là MySQL, PostgreSQL, MongoDB, H2, hoặc các hệ quản trị cơ sở dữ liệu khác.
- **Cách hoạt động:**
 - ☐ Cơ sở dữ liệu sẽ lưu trữ dữ liệu liên quan đến các đối tượng trong model (ví dụ: các yêu cầu bảo hiểm trong bảng Claim).
 - ☐ Các thao tác truy xuất, lưu trữ, sửa đổi, xóa dữ liệu sẽ được thực hiện thông qua lớp Repository.
 - ☐ Tất cả config với database sẽ được config tự động lưu trong file application.properties

```
application.properties x
1 # MySQL configurations
2 spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect
3 spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
4 spring.datasource.url=jdbc:mysql://localhost:3306/insuranceclaim?allowPublicKeyRetrieval=true&useSSL=false
5 spring.datasource.username=root
6 spring.datasource.password=root
```

5) Giải Thích Luồng Đi của Mô Hình Spring Boot

Dependency Injection

- **Vai trò:** Spring Boot sử dụng Dependency Injection (DI) để cung cấp các đối tượng phụ thuộc cho các lớp khác.
- **Cách hoạt động:**
 - DI giúp tách biệt các lớp và tăng tính module của ứng dụng.
 - Ví dụ, khi bạn khai báo `@Autowired` trong Spring Boot, nó tự động inject các bean (ví dụ: `ClaimRepository`, `ClaimService`) vào các lớp mà không cần phải tạo đối tượng thủ công.

```
@RestController  @kiennd25
@RequestMapping("/api/claims")
public class ClaimResourceController {

    @Autowired
    private ClaimService claimService;
```

```
19
20 @Service  @kiennd25
21 public class ClaimServiceImpl implements ClaimService {
22
23     @Autowired
24     private ClaimRepository claimRepository;
```

5) Giải Thích Luồng Đi của Mô Hình Spring Boot

- Tóm lại luồng đi của request trong Spring boot
- 1. **Client gửi yêu cầu:** Client gửi yêu cầu HTTP đến Controller (ví dụ: yêu cầu thông tin yêu cầu bảo hiểm theo ID).
- 2. **Controller nhận yêu cầu:** Controller nhận yêu cầu và gọi Service Layer để xử lý nghiệp vụ.
- 3. **Service Layer xử lý:** Service Layer thực hiện các logic nghiệp vụ, ví dụ như gọi Repository để lấy dữ liệu từ cơ sở dữ liệu.
- 4. **Repository thao tác với cơ sở dữ liệu:** Repository thực hiện các thao tác CRUD trên cơ sở dữ liệu thông qua JPA hoặc Spring Data.
- 5. **Controller trả kết quả:** Sau khi xử lý xong, Service trả kết quả về Controller và Controller trả dữ liệu cho Client dưới dạng JSON hoặc HTML.

6) Các Bước Tạo Một Project Spring Boot

- Bước 1: Tạo Project Truy cập trang Spring Initializr(<https://start.spring.io/>) điền các thông tin như hình. Project của hệ thống quản lý các yêu cầu bồi thường bảo hiểm

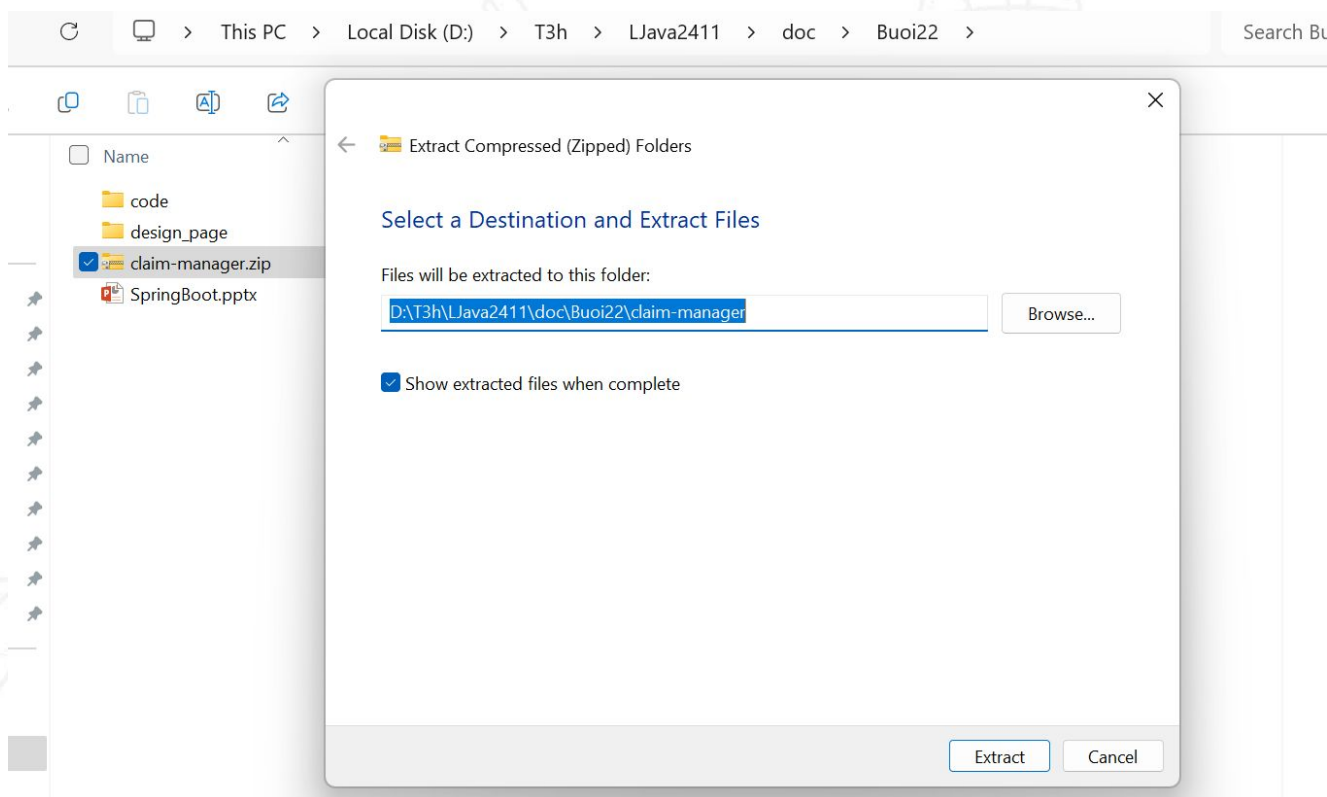
The screenshot shows the Spring Initializr web form with the following configuration:

- Project:** ☒ Gradle - Groovy, ☐ Gradle - Kotlin, ☒ Java, ☐ Kotlin, ☐ Groovy
- Language:** ☒ Java, ☐ Kotlin, ☐ Groovy
- Spring Boot:** ☐ 3.5.0 (SNAPSHOT), ☐ 3.5.0 (M2), ☐ 3.4.4 (SNAPSHOT), ☒ 3.4.3, ☐ 3.3.10 (SNAPSHOT), ☐ 3.3.9
- Project Metadata:**
 - Group: vn.com.t3h
 - Artifact: claim-manager
 - Name: claim-manager
 - Description: System claim manager
 - Package name: vn.com.t3h
 - Packaging: ☒ Jar, ☐ War
 - Java: ☐ 23, ☐ 21, ☒ 17
- Dependencies:**
 - Spring Web** (WEB): Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.
 - Lombok** (DEVELOPER TOOLS): Java annotation library which helps to reduce boilerplate code.
 - Spring Boot DevTools** (DEVELOPER TOOLS): Provides fast application restarts, LiveReload, and configurations for enhanced development experience.
 - Thymeleaf** (TEMPLATE ENGINES): A modern server-side Java template engine for both web and standalone environments. Allows HTML to be correctly displayed in browsers and as static prototypes.

At the bottom, there are buttons: **GENERATE** (CTRL + G), **EXPLORE** (CTRL + SPACE), and a menu button (three dots).

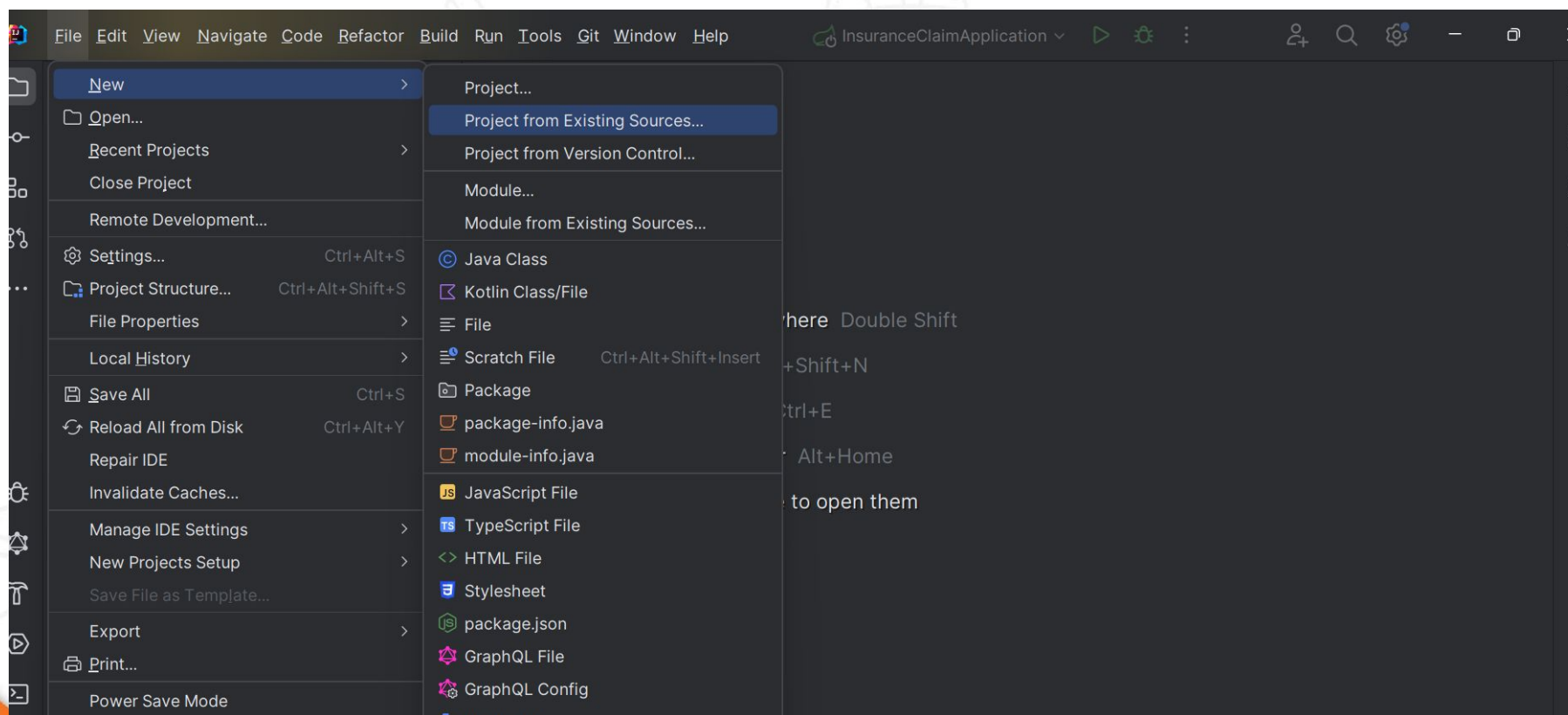
6) Các Bước Tạo Một Project Spring Boot

- Bước 2: Giải nén file zip đã được tải về



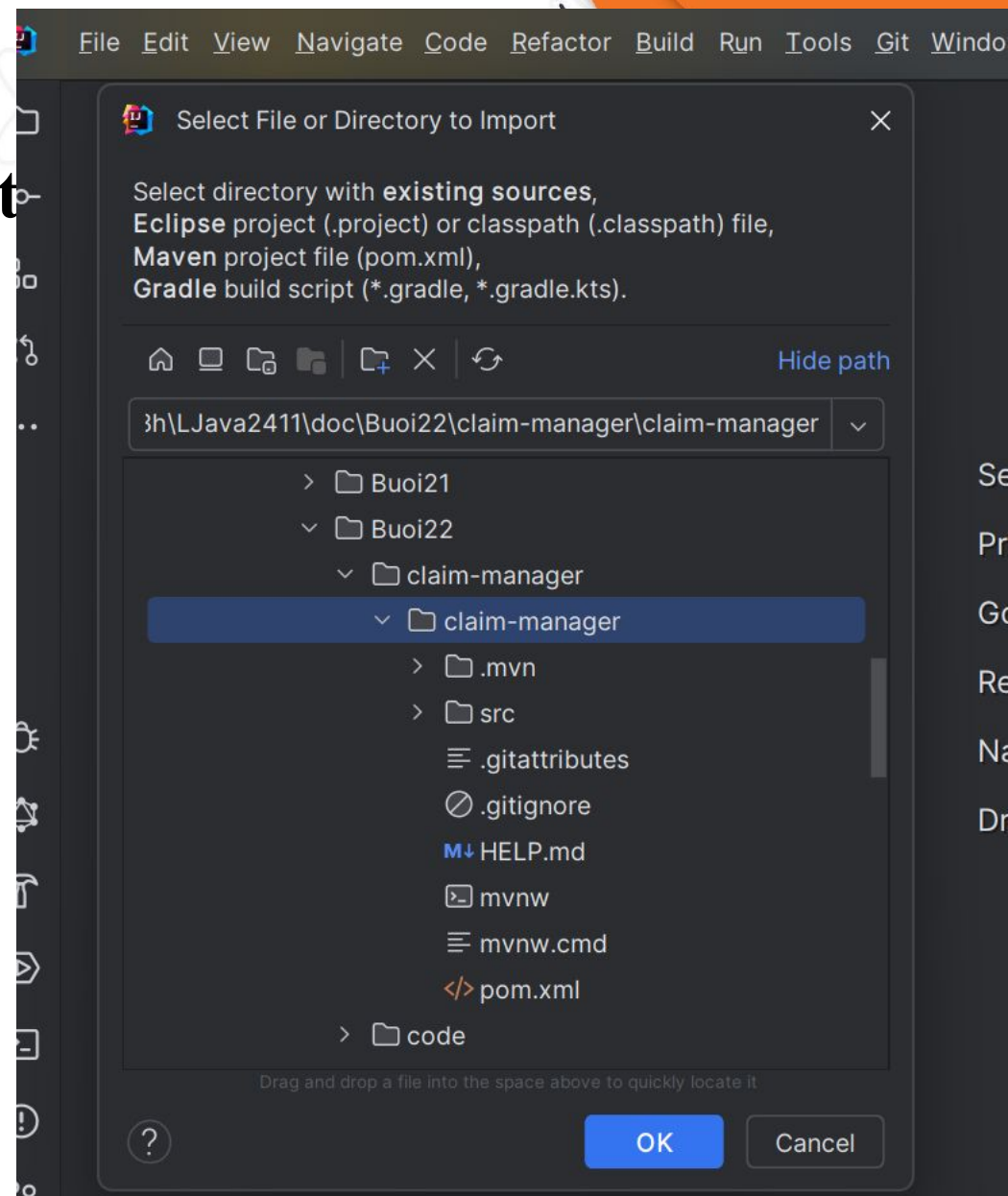
6) Các Bước Tạo Một Project Spring Boot

- B3) chọn chế độ import project đã tồn tại sẵn source



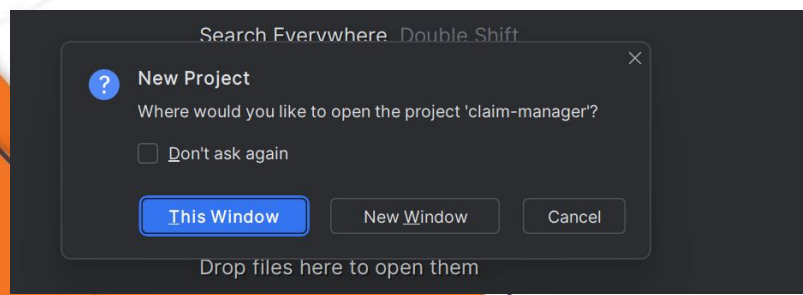
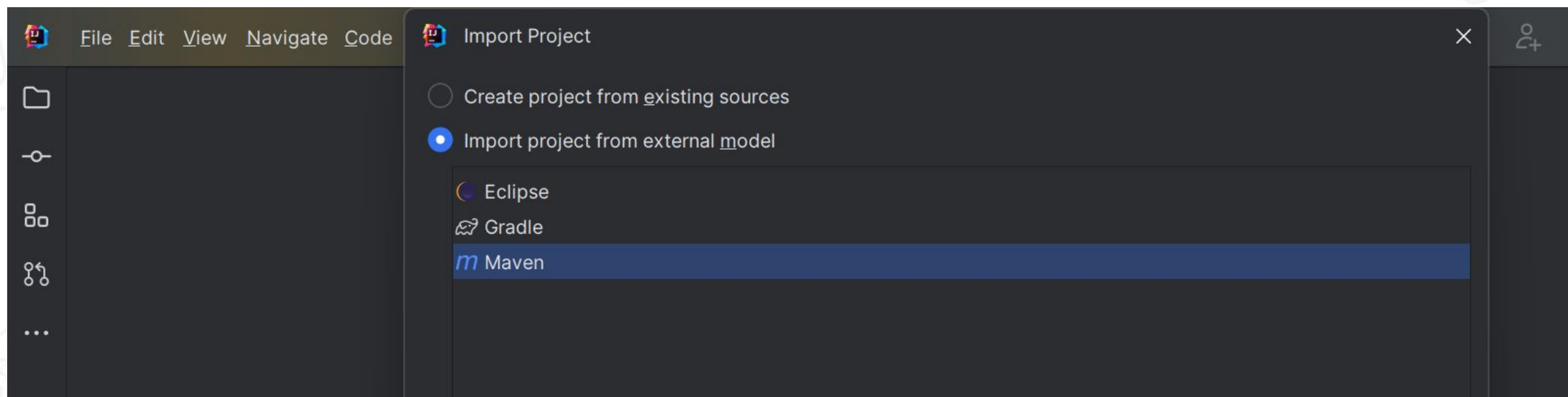
6) Các Bước Tạo Một Project Spring Boot

- B3) Chọn tới folder đã giải nén để import project
- **Lưu ý chọn từ folder cha của folder src. Đó mới là folder gốc của project**



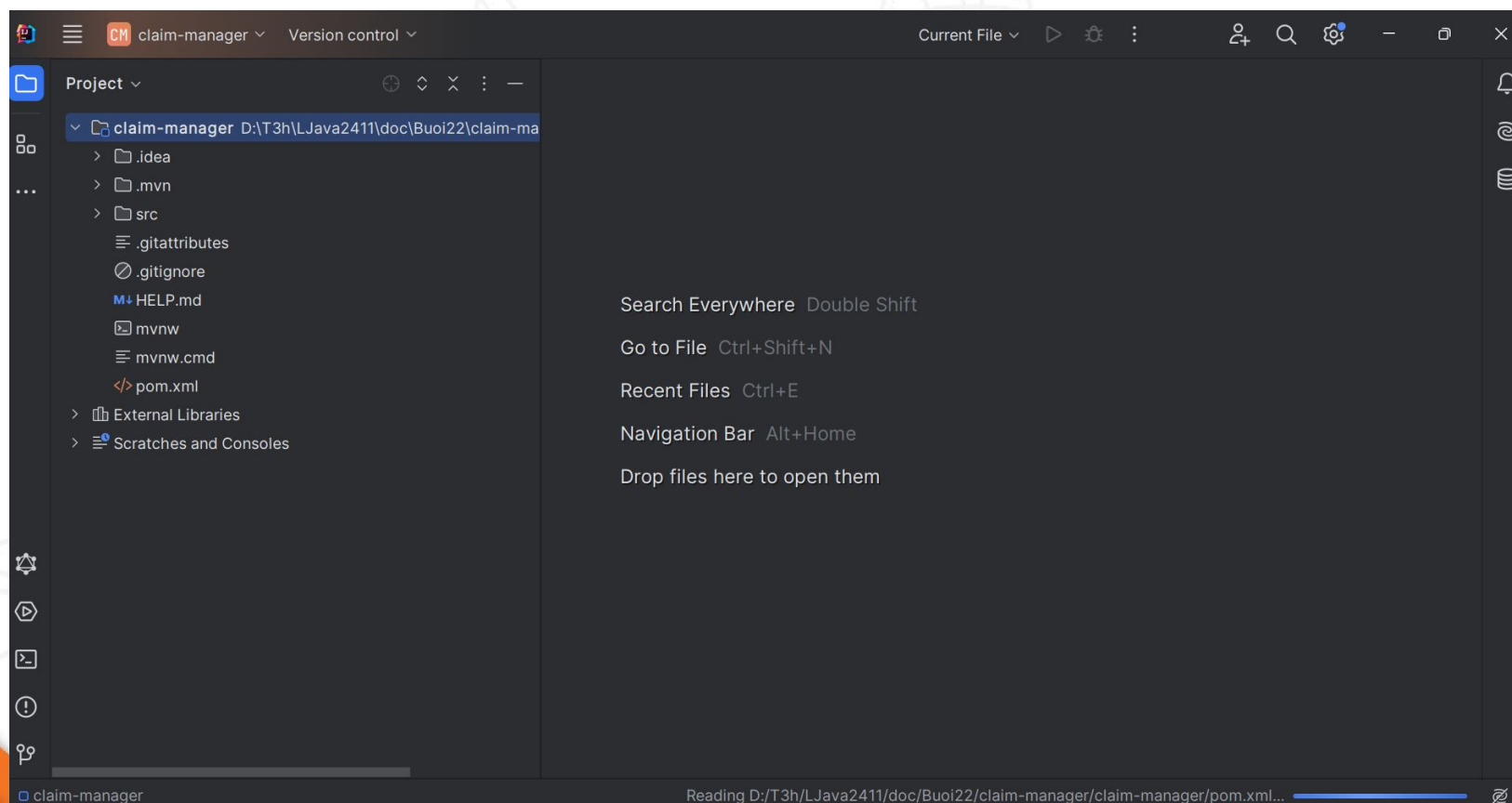
6) Các Bước Tạo Một Project Spring Boot

- B4) Chọn loại công nghệ quản lý thư viện, ở project này đang sử dụng Maven lên sẽ chọn maven, tùy project khác có thể sử dụng Gradle sẽ chọn Gradle. Click Ok và có thể chọn this window hoặc new window



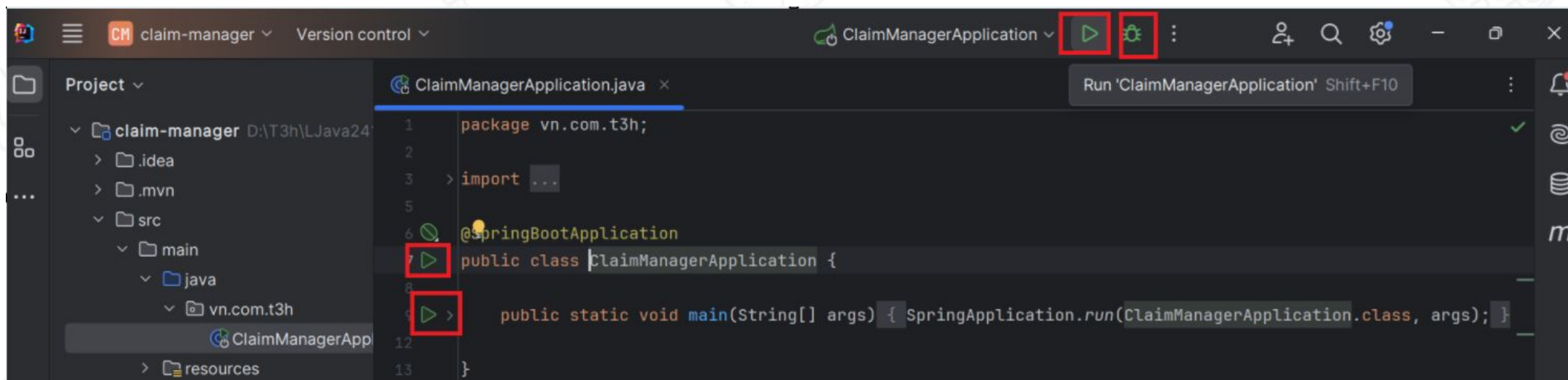
6) Các Bước Tạo Một Project Spring Boot

- B5) Chờ IDE import project và load các thư viện



6) Các Bước Tạo Một Project Spring Boot

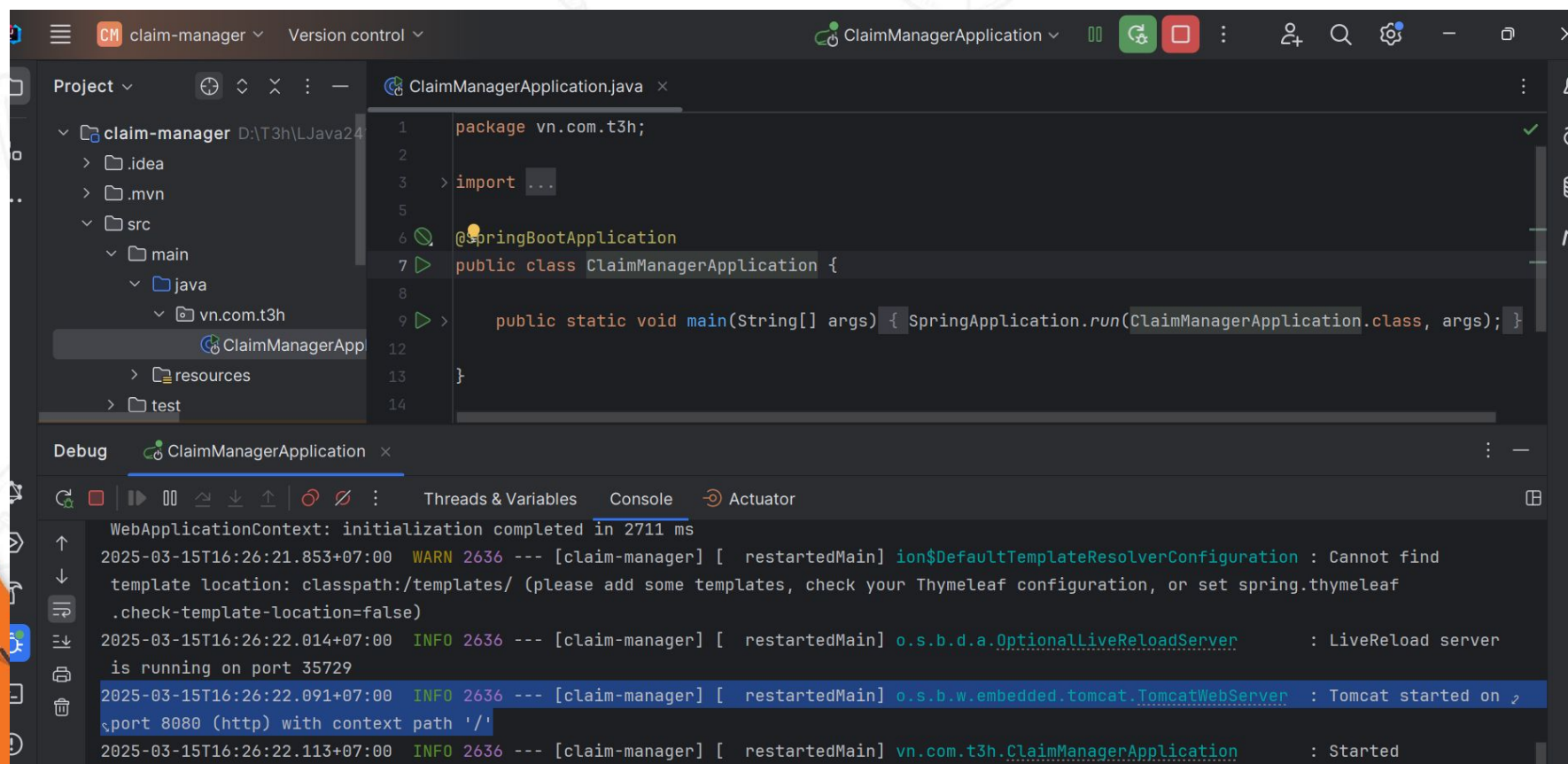
- B6) Sau khi project đã được import thành công sẽ hiển thị class Main để chạy chương trình



- Click vào 1 trong các phần được khoanh đỏ để chạy chương trình

6) Các Bước Tạo Một Project Spring Boot

- B7) Khi chạy chương trình theo dõi phần log console nếu thấy hiển thị Tomcat started on port 8080 như bôi đen trong ảnh tức đã tạo project thành công



```
package vn.com.t3h;

import ...

@SpringBootApplication
public class ClaimManagerApplication {

    public static void main(String[] args) { SpringApplication.run(ClaimManagerApplication.class, args); }

}
```

WebApplicationContext: initialization completed in 2711 ms
2025-03-15T16:26:21.853+07:00 WARN 2636 --- [claim-manager] [restartedMain] ion\$DefaultTemplateResolverConfiguration : Cannot find template location: classpath:/templates/ (please add some templates, check your Thymeleaf configuration, or set spring.thymeleaf.check-template-location=false)
2025-03-15T16:26:22.014+07:00 INFO 2636 --- [claim-manager] [restartedMain] o.s.b.d.a.OptionalLiveReloadServer : LiveReload server is running on port 35729
2025-03-15T16:26:22.091+07:00 INFO 2636 --- [claim-manager] [restartedMain] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path '/'
2025-03-15T16:26:22.113+07:00 INFO 2636 --- [claim-manager] [restartedMain] vn.com.t3h.ClaimManagerApplication : Started

